

Sparse LCS Common Substring Alignment

Gad M. Landau^{a,b,*,1}, Baruch Schieber^c, Michal Ziv-Ukelson^{a,2}

^a Department of Computer Science, Haifa University, Haifa 31905, Israel

^b Department of Computer and Information Science, Polytechnic University, Six MetroTech Center, Brooklyn, NY 11201-3840, USA

^c IBM T.J. Watson Research Center, P.O. Box 218, Yorktown Heights, NY 10598, USA

Received 24 October 2002; received in revised form 9 September 2003

Communicated by S. Albers

Abstract

The “Common Substring Alignment” problem is defined as follows. The input consists of a set of strings $S_1, S_2, \dots, S_c$, with a common substring appearing at least once in each of them, and a target string T . The goal is to compute similarity of all strings S_i with T , without computing the part of the common substring over and over again.

In this paper we consider the Common Substring Alignment problem for the LCS (Longest Common Subsequence) similarity metric. Our algorithm gains its efficiency by exploiting the *sparsity* inherent to the LCS problem. Let Y be the common substring, n be the size of the compared sequences, L_Y be the length of the LCS of T and Y , denoted $|LCS[T, Y]|$, and L be $\max\{|LCS[T, S_i]|\}$. Our algorithm consists of an $O(nL_Y)$ time encoding stage that is executed once per common substring, and an $O(L)$ time alignment stage that is executed once for each appearance of the common substring in each source string. The additional running time depends only on the length of the parts of the strings that are not in any common substring.

© 2003 Elsevier B.V. All rights reserved.

Keywords: Common Substring Alignment; Sparsity; LCS; Algorithms

1. Introduction

The problem of comparing two sequences A and B to determine their similarity is one of the funda-

mental problems in pattern matching. One of the basic forms of the problem is to determine the *longest common subsequence* (LCS) of A and B . The LCS string comparison metric measures the subsequence of maximal length common to both sequences [1]. Longest Common Subsequences have many applications, including sequence comparison in molecular biology as well as the widely used *diff* file comparison program. The LCS problem can be solved in $O(mn)$ time, where m and n are the lengths of strings A and B , using dynamic programming [17]. The dynamic programming creates an $m \times n$ “DP Table” that contains in its (i, j) entry the LCS of the prefix of A of size i and the prefix of B of size j .

* Corresponding author.

E-mail addresses: landau@poly.edu (G.M. Landau), sbar@watson.ibm.com (B. Schieber), michal@cs.haifa.ac.il (M. Ziv-Ukelson).

¹ Partially supported by NSF grant CCR-0104307, by the Israel Science Foundation grant 282/01, by the FIRST Foundation of the Israel Academy of Science and Humanities, and by IBM Faculty Partnership Award.

² On Education Leave from the IBM T.J. Watson Research Center; partially supported by the Israel Science Foundation grant 282/01, and by the FIRST Foundation of the Israel Academy of Science and Humanities.

$T = \text{"BCBADBD CD"}$	$Y = \text{"BCBD"}$	
$S_1 = \text{"BC BCBD C"}$	$B_1 = \text{"BC"}$	$F_1 = \text{"C"}$
$S_2 = \text{"E BCBD D BCBD A"}$	$B_{2a} = \text{"E"}$	$F_{2a} = \text{"D BCBD A"}$
	$B_{2b} = \text{"E BCBD D"}$	$F_{2b} = \text{"A"}$

Fig. 1. An example of two different source strings S_1, S_2 sharing a common substring Y , and a target T .

More efficient LCS algorithms, which are based on the observation that the LCS solution space is highly redundant, exploit in various ways the *sparsity* inherent to the LCS problem. Sparsity means that, for some compared strings, not all of the $n \times m$ entries of the DP Table are actually essential for the LCS computation. Therefore, algorithms that try to limit the computation only to those entries of the DP Table which convey essential information can yield algorithmic performances related to parameters other than the lengths of the input strings. For example, there are applications where most positions of one sequence match relatively few positions in the other sequence, in which case algorithms which utilize a parameter r , defined as the number of positions that match (i.e., the number of DP Table entries $[(i, j) \mid A[i] = B[j]]$), will be superior). This approach was first exploited by the Hunt–Szymanski algorithm [18], which achieves an $O(r \log n)$ complexity. Another approach is demonstrated by the Hirschberg algorithm [17], which uses $L = |\text{LCS}[A, B]|$ as a parameter, and achieves an $O(nL)$ complexity. Apostolico and Guerra [2] achieve an $O(L \cdot m \cdot \min(\log |\Sigma|, \log m, \log(2n/m)))$ algorithm, where $m \leq n$, and another $O(m \log n + d \log(nm/d))$ algorithm, where $d \leq r$ is the number of dominant matches (as defined by Hirschberg [17]). This algorithm can also be implemented in time $O(d \log \log \min(d, nm/d))$ [8]. All sparse LCS algorithms are preceded by an $O(n \log |\Sigma|)$ preprocessing [1]. Note that in the worst case both d and r are $\Omega(n^2)$, while L is always bounded by n .

The *Common Substring Alignment problem* is defined in [22] as follows: The input consists of a set of one or more strings $S_1, S_2, \dots, S_c$ and a target string T . It is assumed that a common substring Y appears in all strings S_i at least once. Namely, each S_i can be decomposed in at least one way to $S_i = B_i Y F_i$. (See Fig. 1.) The goal is to compute the similarity of all strings S_i with T , without computing the part of Y over and over again. It is assumed that the locations of the common subsequence Y in each source sequence S_i are known.

However, the part of the target T with which Y aligns, may vary according to each B_i and F_i combination.

More generally, the common substring Y could be shared by different source strings competing over similarity with a common target, or could appear repeatedly in the same source string. Also, in a given application, we could of course be dealing with more than one repeated or shared subcomponent.

There are various applications that can be cast as Common Substring Alignment problems. The applications differ by the pattern in which the common subcomponents are repeated or shared by the source strings, and therefore may vary in the potential combinatorial gain by applying Common Substring Alignment algorithms to their solution. In “Template Matching Applications” the data is viewed as a set of many competing source sequences to be compared to a common target. The template source sequences are usually known well in advance, and the target is given online. The objective is to classify the target by finding the source string whose alignment with the target gives the highest similarity score. Examples of Template Matching Applications are found in the domains of Intelligent Tutoring [11], Electronic Commerce [5,6,23], and Network Security [30]. Very often, the competing source strings are variations of a similar signal, or different combinations of a common set of subcomponents. Common Substring Alignment can be used to speed up the comparison of each common subcomponent, rather than comparing it again and again for each template source string containing the common subcomponent.

Another application of Common Substring Alignment, called “Repetitive Substring Alignment”, is found in the alignment of two sequences, a source sequence and a target sequence, where the source sequence is known to be highly repetitive. Here, each repeated factor in the source string is, in essence, a common substring which we would like to compare against the target only once during an encoding stage,

rather than comparing it again and again for each appearance of the repeated factor in the source string, during the matching stage. Examples of repetitive sequences are found in genomic data, such as DNA sequences, which are highly repetitive. At the genomic level, repeats are common and crucial components of molecular evolution. Repeats can be classified into two types, long-fragment repeats and short repeats (sometimes tandem) of very short sequences (e.g., dinucleotides or triplets) that appear hundreds and thousands of times in a row. Typically, the length of the long-fragment repeats ranges from a several dozen to a few thousands nucleotides. Although these are not exact repeats, they usually contain runs of exact repeats of at least few dozen nucleotides. It has been noticed that all-together repeats constitute significant portions of genomes from prokaryotes [28] to human [25]. In this context, the *Repetitive Substring Alignment problem* is defined as follows. The inputs are an alphabet Σ , a source string S over the alphabet of the form $S = (\Sigma^* Y \Sigma^*)^*$, and a target string T . The goal is to compute similarity of the full source string S with T , without computing the part of Y again and again. (In a given application, we could of course be dealing with more than one repeated or shared subcomponent.)

Another class of Common Substring Alignment problems is that of “Subcomponent Concatenation With Preserved Order”. In applications belonging to this category, the source string is segmented into many subparts, and the target string is matched against different concatenations of these source substrings. The concatenations preserve the ordering of the subsegments in the source string. Gene Prediction Via Spliced Alignment [14,24] is an example of an application from this category.

For a detailed description of some additional applications for the problem see [22].

Common Substring Alignment algorithms are usually composed of a *preprocessing* stage that depends on data availability, an *encoding* stage and an *alignment* stage.

During the preprocessing stage, the source strings are parsed into their optimal common substring representation. It is during this stage that the best-covering common substrings are chosen, and for each selected common substring Y it is decided where, in each S_i , Y begins and ends. During the encoding stage, a data structure is constructed which encodes the compari-

son of Y with T . Then, during the alignment stage, for each comparison of a source S_i with T , the pre-compiled data structure is used to speed up the part of aligning each appearance of the common substring Y .

In most of the applications for which Common Substring Alignment is intended, the source sequence database is prepared *off-line*, while the target can be viewed as an “unknown” sequence which is received *online*. Therefore, the pre-processing stage can be executed *off-line*. However, the comparison of Y and T can not be computed until the target is received. Therefore, the encoding stage, as well as the alignment stage, are both *online* stages.

Even though both the encoding and the alignment stages are *online*, they do not bear an equal weight on the time complexity of the algorithm. The efficiency gain is based on the fact that the encoding stage is executed only once per target, and then the encoding results are used, during the alignment stage, to speed up the alignment of each appearance of the common substring in any of the source strings.

To simplify notation we assume from now on that all compared strings are of size n . Our results can be extended easily to the case in which the strings are of different length. We use L to denote $\max\{|\text{LCS}[T, S_i]|\}$, and L_y to denote $|\text{LCS}[T, Y]|$. (Note that $L_y \leq |Y|$, $L_y \leq L$, and $L \leq n$.)

Results. In this paper we address the following challenge: can a more efficient common substring alignment algorithm, which exploits the *sparsity* inherent to the LCS metric, be designed for the LCS metric. We show how to exploit sparsity, by replacing the traditional matrix which is used to encode the comparison of Y and T , with a smaller matrix. We show that this smaller matrix can be computed using Myers’ Consecutive Suffix Alignments algorithm [27]. We also prove that this smaller matrix preserves the Total Monotonicity condition, thus enabling efficient adaptation of a matrix searching algorithm of Aggarwal et al. [3].

The first Common Substring Alignment algorithm for the LCS metric was given in [21]. It presents an $O(n^2 + n|Y|)$ encoding stage, and an $O(n)$ alignment stage. In [22] a Common Substring Alignment algorithm for the LCS, Edit Distance and more extended metrics is given. This algorithm consists of an $O(n|Y|)$ encoding stage, and an $O(n)$ alignment stage.

The algorithm which is presented in this paper consists of an $O(nL_y)$ encoding stage, and an $O(L)$ alignment stage. When the problem is sparse ($L_y \ll |Y|$, $L \ll n$), our time bounds are better than those of our previous algorithms [21]. Even when the data is dense, our solution for the problem is no worse than the best-known algorithms.

The remainder of this paper is organized as follows. Section 2 contains Common Substring Alignment preliminaries. The new algorithm is described in Section 3. Section 4 contains an analysis and assertion of some of the properties of the new data representation which allow for the efficiency gain. Conclusions and open problems are given in Section 5.

2. Common Substring Alignment preliminaries

In the literature the DP Table used for computing the alignment is also viewed as a directed acyclic graph (DAG), called the Dynamic Programming (DP) Graph [16]. The DP Graph for S and T contains $(|S| + 1)(|T| + 1)$ vertices, each labeled with a distinct pair (x, w) ($0 \leq x \leq |S|$, $0 \leq w \leq |T|$). The vertices are organized in a matrix of $(|S| + 1)$ rows and $(|T| + 1)$ columns. (See Fig. 2.)

When using the LCS metric, the DP Graph contains a directed edge with weight zero from each vertex (x, w) to each of the vertices $(x, w + 1)$, $(x + 1, w)$. It also contains a diagonal edge with weight one from vertex (x, w) to vertex $(x + 1, w + 1)$ if $S[x + 1] = T[w + 1]$. Maximal weight paths in the Dynamic Programming graph represent optimal alignments of S and T . The Dynamic Programming algorithm will set the value of vertex (i, j) in the graph to the total weight of the highest scoring path which originates in vertex $[0, 0]$ of the graph and ends in vertex $[i, j]$.

The Dynamic Programming Graph used for computing the similarity between a source string $S_i = B_i Y F_i$ and a target string T can be viewed as a concatenation of 3 subgraphs, where the first subgraph represents the similarity between B_i and T , the second subgraph represents the similarity between Y and T , and the third subgraph represents the similarity between F_i and T .

In this partitioned solution, the weights of the vertices in the last row of the first graph serve as input to initialize the weights of the vertices in the first row

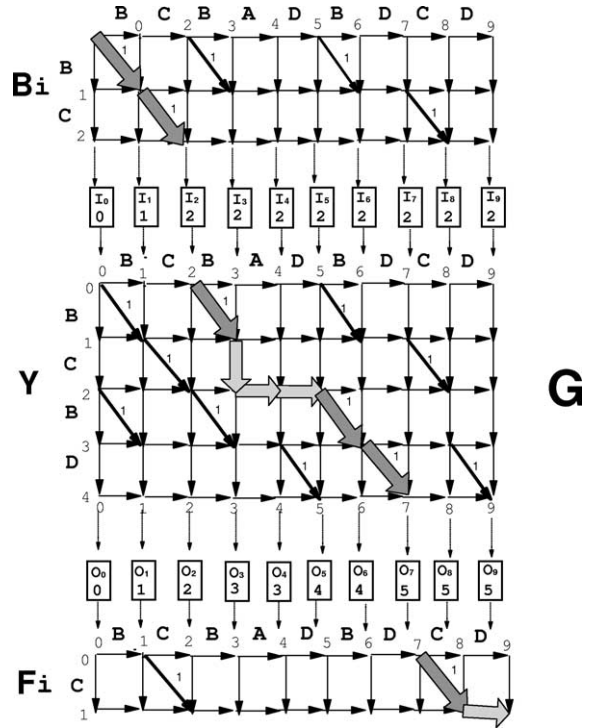


Fig. 2. The LCS Dynamic Programming Graph for the comparison of $S_1 = "BCBCBDC"$ with $T = "BCBADBCDC"$. The highlighted path in the graph corresponds to the common subsequence "BCBBDC".

of the second graph. The weights of the last row of the second graph can be used to initialize the first row of the third graph.

The motivation for breaking the solution into 3 subgraphs is that the second subgraph, which represents the comparison of Y with T , is identical in each of its appearances in all DP graphs comparing any of the strings S_i with T . More specifically, both the structure and the weights of the edges of all DP subgraphs comparing Y with T are identical, but the weights to be assigned to the vertices during the LCS computation may vary according to the prefix B_i which is specific to the source string. Therefore, an initial investment in the learning of this graph as an encoding stage, and in its representation in a more informative data structure, may pay off later on.

For a string A of length n and for $1 \leq u \leq z \leq n$, let A_u^z denote the substring of A from index u (inclusive) up to index z (inclusive). We define the following notations with respect to the Dynamic Programming

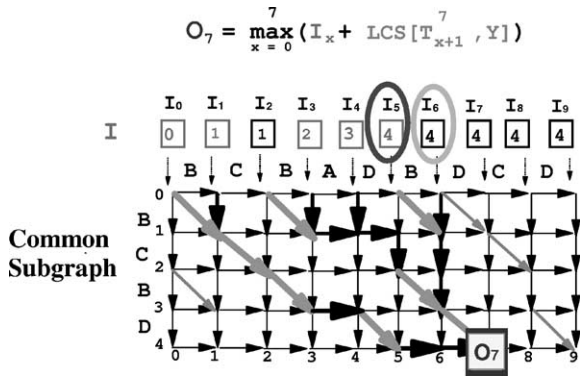


Fig. 3. The computation of output entry O_7 for $T = \text{"BCBADBDCD"}$, $B_i = \text{"CBA"}$, and $Y = \text{"BCBD"}$. Note that the highest scoring path from $(0, 0)$ to O_7 enters G through I_5 , which yields the score $I_5 + \text{LCS}[T_6^7, Y] = 4 + 2 = 6$.

Graph used for computing the LCS of $S_i = B_i Y F_i$ and T . Let G denote the second subgraph which represents the comparison of Y with T . Let I denote the first row of G . Namely, $I[j] = |\text{LCS}[T_1^j, B_i]|$, for $j = 1, \dots, n$. Let O denote the last row of G . Namely, $O[j] = |\text{LCS}[T_1^j, B_i Y]|$, for $j = 1, \dots, n$.

Our solution will focus on the work necessary for an appearance of a given common substring: Given a target string T , a source substring Y and an input row I , compute the output row O .

As described above the online work for each common substring consists of two stages.

- (1) *Encoding stage*: Study the structure of G and represent it in an efficient way.
- (2) *Alignment stage*: Given I and the encoding of G , constructed in the previous stage, compute O .

Note that each entry at given index j of the input row, denoted I_j , represents the LCS of B_i and a prefix of size j of the target. $I_j = \text{LCS}[T_1^j, B_i]$, $I_{j+1} = \text{LCS}[T_1^{j+1}, B_i]$. T_1^j is a prefix of T_1^{j+1} and therefore, by definition, any subsequence common to T_1^j and B_i will also be a subsequence of T_1^{j+1} . However, $\text{LCS}[T_1^{j+1}, B_i]$ can either stay the same as $\text{LCS}[T_1^j, B_i]$ or increase by one with the additional character in T_1^{j+1} . The same observations apply similarly to the output row series O . Thus, both I and O are monotone staircases with at most $L + 1$ unit steps.

This enables the following definitions of the vectors PI and PO , which will represent I and O , correspondingly.

Definition 1. For $k = 0, \dots, L$, the entry $PI[k]$ contains the smallest index in the input row I whose value is k , if such exists.

Definition 2. For $k = 0, \dots, L$, the entry $PO[k]$ contains the smallest index in the output row O whose value is k , if such exists.

We claim that only the positions $PI[r]$ for $r = 0, \dots, L$ are interesting as intermediate points in highest-scoring paths from vertex $(0, 0)$ to any entry of O . (The compressed representation of these highest-scoring paths as leftmost k -paths ending in PO will be discussed in the next section.) To demonstrate this we use the example of Fig. 3. Consider the two entries I_5 and I_6 in the input row as demonstrated in this example. Both entries fall in the fourth plato of the input row staircase, and I_5 is the fourth step in the input series. We will compare the contribution of the two input row entries I_5 and I_6 in the context of the computation of output row entry O_7 ,

$$O_7 = \max_{x=0}^7 \{I_x + |\text{LCS}[T_{x+1}^7, Y]|\}.$$

Since $I_6 = I_5 = 4$ and $|\text{LCS}[T_6^7, Y]| \geq |\text{LCS}[T_7^7, Y]|$, it follows that I_6 is redundant. More generally we conclude that, for the computation of any output row entry, the k th step in the input series staircase is sufficient to represent all other input entries in the k th plato, and therefore all non-step input entries are redundant.

3. The algorithm

The objective of the algorithm is to compute $PO[k]$, for $k = 1, \dots, L$ given the vector PI . (Note that $PO[0] = 0$.) Recall that $PO[k]$ is the smallest index of an entry in O with a value of k . In terms of optimal paths in the alignment graph, this means that $PO[k]$ is the index of the leftmost vertex in the output border of G to end a path of weight k , which originates in vertex $(0, 0)$ of the alignment graph. Note that such a path could enter G through any one of its

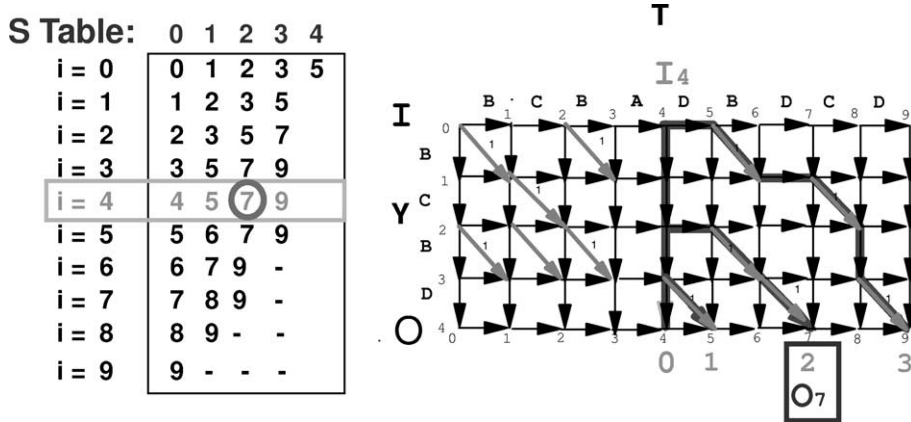


Fig. 4. The table S that is computed during the encoding stage, for the common subgraph which corresponds to strings $Y = \text{"BCBD"}$ and $T = \text{"BCBADBDCD"}$. $S[i, k]$ is set to the index of the leftmost vertex in the last row of the common subgraph that can be reached from vertex i in the first row via a path of size k . Highlighted in the common subgraph grid are the leftmost ending paths of sizes $0 \dots 3$ from I_4 to the last row of the graph, which correspond to the values in row 4 of S . For example, $S[4, 2] = 7$, since O_7 is the leftmost vertex in the last row of the common subgraph that can be reached from I_4 via a path of size 2. This path reflects the common substring "BD".

input border vertices whose value is $\leq k$. However, in Section 2 we have shown that the PI indices are sufficient for representing all the potential entry points of I , and therefore only the values $PI[0, \dots, k]$ will be considered as relevant representative entry point indices for the sought k -path.

Now, consider any path of total weight k , which originates in vertex $(0, 0)$ of the alignment graph, enters G through a given input border vertex $PI[r]$, and ends in some vertex j on the output border of G . This path could be decomposed into two parts: the subpath connecting vertex $(0, 0)$ with the selected input border entry $PI[r]$, followed by the subpath from the selected input border entry to vertex j on the output border (see Fig. 5(A)). The weight of the subpath from vertex $(0, 0)$ to the selected input border entry is the value $I_{PI[r]} = r$. The weight of the subpath from vertex $PI[r]$ on the input border to vertex j on the output border is $|\text{LCS}[T_{PI[r]+1}^j, Y]|$. By definition, the sum of the weights of these two subpaths, whose concatenation gives the total k -path, must be k .

Now recall that among all such potential k -paths, which could enter G through any of the $PI[r]$, $r = 0, \dots, k$ input border vertices, we are actually interested in the ones which are optimal in the sense that they end in the leftmost possible output border vertex.

Therefore, the value of $PO[k]$, for $1 \leq k \leq L$, is computed as follows.

$$PO[k] = \min_{r=0}^k \{j \mid r + |\text{LCS}[T_{PI[r]+1}^j, Y]| = k\}. \quad (1)$$

We show how to obtain PO in two stages.

- (1) The encoding stage which is executed only once for each common substring Y , in which we compute $\min\{j \mid |\text{LCS}[T_{i+1}^j, Y]| = k\}$ for each pair (i, k) , $i = 0, \dots, n$ and $k = 1, \dots, L_y$.
- (2) The alignment stage in which we compute PO , using Eq. (1), given PI and the values computed in the encoding stage.

3.1. The encoding stage

In the encoding stage we compute the $n \times L_y$ table S , where

$$S[i, k] = \min\{j \mid |\text{LCS}[T_{i+1}^j, Y]| = k\},$$

if such an index exists. In other words, $S[i, k]$ contains the smallest index of a vertex in the last row of the DP Graph defined by the LCS of T and Y that can be reached from vertex i in the first row of this graph via a path of weight k .

Note that, in our model, a single encoding stage, computed once, may serve various future alignment

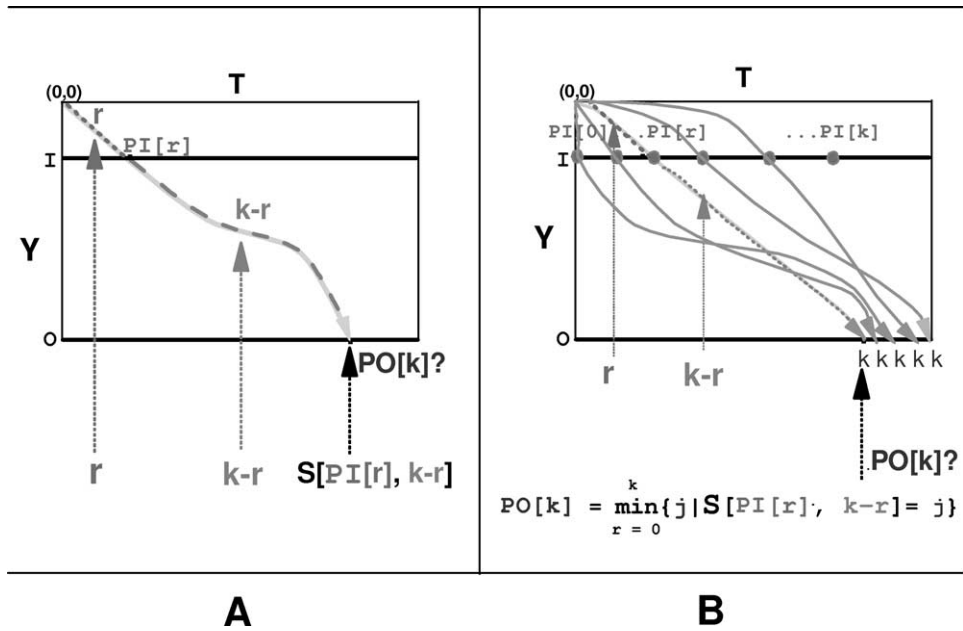


Fig. 5. The computation of $PO[k]$ during the alignment stage, given the table S which was computed during the encoding stage, and the vector PI . Fig. 5(A) demonstrates the computation of $PO[k]$ when it is known in advance that the optimal, leftmost ending path enters G through $PI[r]$. Fig. 5(B) demonstrates the computation of $PO[k]$ when it is not known in advance through which of the PI s the leftmost k -path will enter, and therefore $PO[k]$ will be chosen as the leftmost among all k -paths to O which could enter through any of the first k PI s.

stages, each of which could have a different input series, the partition-points of which are not known in advance. Therefore, the encoding stage must prepare for all possible future input staircases, and each point in the input row must be considered as a potential future PI step.

The *Consecutive Suffix Alignment* problem is, given two sequences T and Y , compute the DP tables for the comparisons of each suffix of T versus the full string Y . We observe that Myers' Consecutive Alignments algorithm [27] can be used to construct the table S .

Complexity. Given two strings Y and T over a constant alphabet Myers [27] constructs the table S for the comparison of Y versus T in $O(nL_Y)$ time and space.

3.2. The alignment stage

During each execution of the alignment stage the objective is to compute PO from PI , using the table S computed in the encoding stage.

Suppose we were to know in advance that the leftmost-ending k -path from $(0,0)$ to O enters G through $PI[r]$. In this case, $PO[k]$ could be easily computed as $S[PI[r], k-r]$ (see Fig. 5(A)). However, since we do not know in advance through which of the active entry points in PI the sought path will enter G , each one of the first k partition-points of I should be considered as a relevant entry-point candidate, and so Eq. (1) could be alternatively casted as follows (see Fig. 5(B)).

$$PO[k] = \min_{r=0}^k \{ S[PI[r], k-r] \}. \quad (2)$$

Note that when the alignment stage is executed, the L values of the PI for this specific alignment stage are known, and S has already been computed. The main question, which is key to the efficiency of the alignment stage, is therefore: how many values of S need to actually be queried at this stage?

In order to allow for a precise answer to this question, we need to define two additional parameters. One is $L_I = LCS[B_i, T] \leq L$, which is the number of entries in the PI for the i th alignment stage. The second parameter is $L_O = LCS[B_i, Y, T] \leq \min(L_I +$

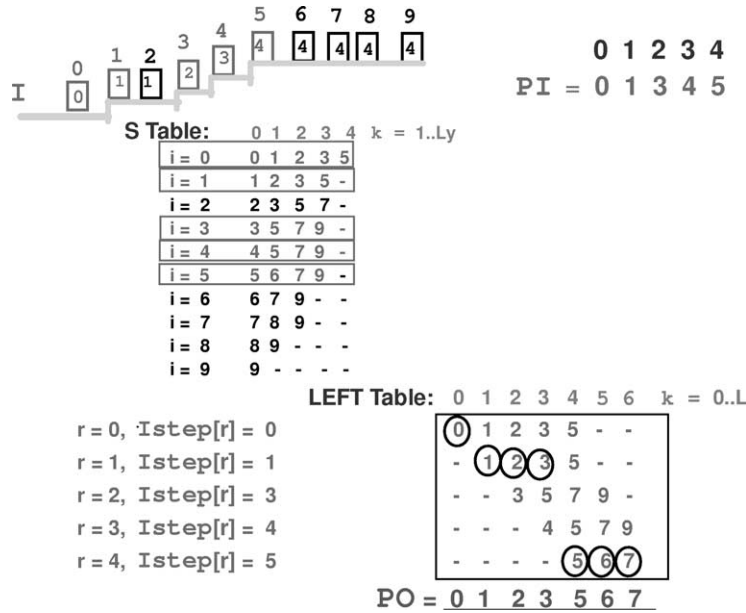


Fig. 6. The computation of PO , using PI and the precomputed S table, which correspond to the sequences $T = "BCBADBDCD"$, $B_i = "CBA"$, and $Y = "BCBD"$. Also shown is the corresponding virtual LEFT table, which is composed of the subset of the rows of S that includes only the rows which correspond to steps in I , shifted to the right by the step value. The selected column minima values are circled in the LEFT table.

$L_y, L)$, which is the number of entries in the PO for the i th alignment stage.

At first glance it seems that $O(L_O \times L_I)$ entries of S need to be queried in order to compute all values of PO . This, due to the fact that we compute $PO[k]$ for $k = 1, \dots, L_O$, and for each entry in PO we compute the minimum of up to $k \leq L_I$ values queried from S , using Eq. (2).

Suppose we were to store all the values which are to be queried from S during a given alignment stage, in a matrix denoted LEFT (see Fig. 6). Column k of LEFT, for $1 \leq k \leq L_O$, stores the values queried from S during the computation of $PO[k]$ via Eq. (2). Therefore, $PO[k]$ is the minimum of the k th column in LEFT. More formally, $LEFT[r, k] = S[PI[r], k - r]$, for $0 \leq r \leq L_I$, so row r of LEFT is row $PI[r]$ of S , shifted to the right by r columns.

In Section 4 we prove that LEFT is convex totally monotone. Hence, a recursive algorithm by Aggarwal et al. [3], nicknamed SMAWK in the literature, can be used to compute the column minima of LEFT, and this algorithm queries only $O(L_O)$ entries of S .

For various solutions to related problems, which also utilize Monge and Total Monotonicity properties,

we refer the interested reader to [4,7,9,10,12,13,19,20,22]. A thorough survey on the subject is given in [15].

Complexity. Given S and PI , computing an element of LEFT requires $O(1)$ time and space. The SMAWK algorithm computes the column minima of the $(L + 1) \times L$ totally monotone matrix LEFT in $O(L)$ time and space, by querying only $O(L)$ entries of the array. Hence, PO is computed during the alignment stage in $O(L)$ time and space.

4. LEFT as a totally monotone rectangular matrix

Definition 3. A matrix $M[0, \dots, m, 0, \dots, n]$ is *totally monotone* if either condition (1) or (2) below holds for all $a, b = 0, \dots, m; c, d = 0, \dots, n$:

- (1) *convex condition:* $M[a, c] \geq M[b, c] \Rightarrow M[a, d] \geq M[b, d]$ for all $a < b$ and $c < d$.
- (2) *concave condition:* $M[a, c] \leq M[b, c] \Rightarrow M[a, d] \leq M[b, d]$ for all $a < b$ and $c < d$.

In this section we prove that LEFT can be safely transformed into a full, rectangular, convex totally

monotone matrix, as needed for the implementation of SMAWK algorithm in the alignment stage. We start by noting that, originally, LEFT is not a full, rectangular matrix, since some of its entries are undefined. Recall that $\text{LEFT}[r, k]$ is defined as $S[\text{PI}[r], k - r]$. Since I is a monotone staircase, $\text{PI}[r]$ is defined for $0 \leq r \leq L_I$, and undefined for $r > L_I$. We consider only the first L_I rows of LEFT and thus we may assume that $\text{PI}[r]$ is always defined. It follows that an entry $\text{LEFT}[r, k]$ is undefined whenever the entry $S[\text{PI}[r], k - r]$ is undefined.

Definition 4. Let k_r , for $0 \leq r \leq L_I$, denote the greatest column index of an entry in row r of LEFT whose value is defined.

Lemma 1. Each row in LEFT consists of a (possibly empty) span of undefined entries, called the undefined prefix, followed by a span of defined entries, and by a (possibly empty) span of undefined entries, called the undefined suffix. For a given row r of LEFT, the span of defined entries starts in entry $\text{LEFT}[r, r]$ and ends in some entry $\text{LEFT}[r, k_r]$, such that $r \leq k_r \leq L$.

Proof. Suppose that for $c < e$, both $\text{LEFT}[r, c]$ and $\text{LEFT}[r, e]$ are defined. From the definition of the table S it follows that all the entries $S[\text{PI}[r], d - r]$, for $c \leq d \leq e$ are also defined and thus also $\text{LEFT}[r, d]$. This means that the defined entries in each row of LEFT form a consecutive interval. Following the definition of the S and LEFT tables, $S[\text{PI}[r], 0]$ is always defined and therefore $\text{LEFT}[r, r] = S[\text{PI}[r], r - r]$ is always defined. $S[\text{PI}[r], -1]$, on the other hand, is never defined and therefore $\text{LEFT}[r, r - 1]$ is never defined. Thus we conclude that the span of consecutive defined entries in row r of LEFT begins in $\text{LEFT}[r, r]$ and ends in $\text{LEFT}[r, k_r]$. $\square$

Our next goal is to prove that the defined entries of LEFT follow the convex total monotonicity property. Later, we show how to complement the undefined entries of LEFT, without changing its column minima, and still maintain this property.

Lemma 2. For any a, b, c , such that $a < b$ and all four entries: $\text{LEFT}[a, c]$, $\text{LEFT}[b, c]$, $\text{LEFT}[a, c + 1]$, and $\text{LEFT}[b, c + 1]$ are defined, if $\text{LEFT}[a, c] \geq \text{LEFT}[b, c]$, then $\text{LEFT}[a, c + 1] \geq \text{LEFT}[b, c + 1]$.

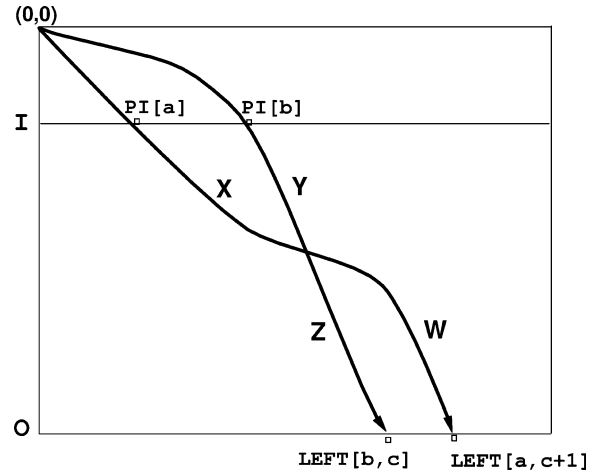


Fig. 7. Shown in the figure are two leftmost-ending paths from $(0, 0)$ to O : the leftmost path of size c which enters G through $\text{PI}[b]$, and the leftmost path of size $c + 1$ which enters G through $\text{PI}[a]$. Note that the two paths must cross.

Proof. The proof is based on a crossing paths contradiction [4,19,26,29].

We consider two paths in the DP Graph of $B_i Y$ and T . Let path A_{c+1} denote an optimal path (of weight $c + 1$) connecting vertex $(0, 0)$ of the graph with vertex $\text{LEFT}[a, c + 1]$ of O and going through vertex $\text{PI}[a]$ of I . Note that by our definition such a path always exists. Similarly, let path B_c denote an optimal path connecting vertex $(0, 0)$ of the graph with vertex $\text{LEFT}[b, c]$ of O and going through vertex $\text{PI}[b]$ of I . Fig. 7 shows paths B_c and A_{c+1} . Note that, since $\text{LEFT}[b, c] \leq \text{LEFT}[a, c] < \text{LEFT}[a, c + 1]$, the two paths B_c and A_{c+1} must intersect at some column of the DP Graph before or at column $\text{LEFT}[b, c]$. Let X and Y be the prefixes of A_{c+1} and B_c up to the intersection point, and let W and Z be their suffixes from this intersection point.

There are two cases to be addressed, depending on the outcome of the comparison of the weights of X and Y , denoted $|X|$ and $|Y|$.

Case 1. $|X| \leq |Y|$. Then $|Y| + |W| \geq c + 1$. By monotonicity of LCS this implies that $\text{LEFT}[b, c + 1] \leq \text{LEFT}[a, c + 1]$, in agreement with the right side of the Total Monotonicity condition.

Case 2. $|X| > |Y|$. Then $|X| + |Z| > c$. By monotonicity of LCS this implies that $\text{LEFT}[a, c] < \text{LEFT}[b, c]$, in contradiction to the assumption of the proof. $\square$

Note that the two assertions of Lemmas 1 and 2 lead, by definition, to the following conclusion.

Conclusion 1. The defined entries of LEFT follow the convex total monotonicity property.

We now turn to handle the undefined entries of LEFT. By Lemma 1 we know that the span of defined entries in a given row r starts in $\text{LEFT}[r, r]$ and ends in $\text{LEFT}[r, k_r]$. This implies, by definition, that the undefined prefix of row r consists of its first $r - 1$ entries, and thus the undefined prefixes form a lower left triangle in LEFT.

The next lemma assists in defining the shape formed by the undefined suffixes of the rows of LEFT.

Lemma 3. For any two rows a, b of LEFT, where $a < b$, if $k_a > k_b$, then $\text{LEFT}[a, \ell] \leq \text{LEFT}[b, \ell]$, for all the defined elements in rows a and b of LEFT.

Proof. Since $k_a > k_b$, it follows that $\text{LEFT}[a, k_b]$, $\text{LEFT}[b, k_b]$ and $\text{LEFT}[a, k_b + 1]$ are defined, yet $\text{LEFT}[b, k_b + 1]$ is undefined. Note that for every index $b \leq j \leq k_b$ both $\text{LEFT}[b, j]$ and $\text{LEFT}[a, j]$ are defined. Suppose there was some index $b \leq j \leq k_b$ such that $\text{LEFT}[a, j] > \text{LEFT}[b, j]$.

We consider two paths in the DP graph of $B_i Y$ and T . Let path A_{k_b+1} denote an optimal path (of weight $k_b + 1$) connecting vertex $(0, 0)$ of the graph with vertex $\text{LEFT}[a, k_b + 1]$ of O and going through vertex $PI[a]$ of I . Note that by our definition such a path always exists. Similarly, let path B_j denote an optimal path (of weight j) connecting vertex $(0, 0)$ of the graph with vertex $\text{LEFT}[b, j]$ of O and going through vertex $PI[b]$ of I .

Similarly to the proof of Lemma 2, one can show that these two paths cross and that since $\text{LEFT}[a, j] > \text{LEFT}[b, j]$ there must be a path of weight at least $k_b + 1$ that originates in vertex $(0, 0)$, ends in O and goes through vertex $PI[b]$ of I , in contradiction to the assumption that $\text{LEFT}[b, k_b + 1]$ is undefined. We conclude that, for all the defined elements in rows a and b of LEFT, $\text{LEFT}[a, j] \leq \text{LEFT}[b, j]$. $\square$

Lemma 3 implies that, for any row r of LEFT, if there exists another row r' in LEFT, such that $r' < r$ and $k_r < k_{r'}$, then row r can be skipped in the search

for column minima. After removing all such rows we are guaranteed that the undefined suffixes form an upper right triangle in LEFT.

The undefined entries of LEFT can be complemented in constant time each, similarly to the solution described in [7], as follows.

- (1) *Lower Left Triangle:* These entries can be complemented by setting the value of any $\text{LEFT}[r, j]$ in the missing lower-left triangle to $(n + r + 1)$.
- (2) *Upper Right Triangle:* The value of any undefined entry $\text{LEFT}[r, j]$ in this triangle can be set to ∞ .

In the next lemma we prove that LEFT can be safely converted into a full, rectangular, convex totally monotone matrix.

Lemma 4. The LEFT matrix can be transformed into a full, rectangular matrix that is totally monotone, without changing its column minima.

Proof. We have shown (Conclusion 1) that the defined entries of LEFT follow the convex total monotonicity property. It remains to show that complementing the undefined entries of LEFT, as described above, preserves its column minima, and still maintains this property.

- (1) *Lower Left Triangle:* The greatest possible index value in LEFT is n . Since r is always greater than or equal to zero, the complemented values in the lower left triangle are lower-bounded by $(n + 1)$ and no new column minima are introduced. Also, for any complemented entry $\text{LEFT}[b, c]$ in the lower left triangle, $\text{LEFT}[a, c] < \text{LEFT}[b, c]$ for all $a < b$, and therefore the convex total monotonicity condition holds.
- (2) *Upper Right Triangle:* All scores in LEFT are finite. Therefore, no new column minima are introduced by the re-defined entries.

Due to the upper right corner triangular shape of this ∞ -patched area, for any two rows a, b , where $a < b$, if $\text{LEFT}[b, c] = \infty$, then surely $\text{LEFT}[a, c] = \infty$. Let d be the smallest index such that $\text{LEFT}[a, d] = \infty$. It follows that $\text{LEFT}[a, e] \geq \text{LEFT}[b, e]$ for all $e \geq d$, and the convex total monotonicity property is preserved. $\square$

4.1. Time complexity

We show that the overhead associated with the marking of the undefined elements in LEFT, as well as the removal of redundant rows in LEFT in order to obtain the crisp undefined upper right triangle, does not slow down the time complexity of the suggested Common Substring Alignment algorithm.

- (1) *Encoding Stage.* During the encoding stage, $\text{LCS}[T_{r+1}^n, Y]$, for $r = 0, \dots, n - 1$ is computed and stored. This information will be used later, during the alignment stage, to mark the beginning of the undefined suffix in each row. $\text{LCS}[T_{r+1}^n, Y]$ corresponds to the greatest column index of a defined entry in row r of S . Therefore, all n values of $\text{LCS}[T_{r+1}^n, Y]$, for $r = 0, \dots, n - 1$, can be queried from the constructed S table without changing the original $O(nL_y)$ complexity of the encoding stage algorithm.
- (2) *Alignment Stage.*
 - (a) *Marking the defined and undefined intervals in each row of LEFT.* Given a row index r and the value $\text{LCS}[T_{r+1}^n, Y]$ which was computed in the encoding stage, the undefined prefix and suffix of row r of LEFT can each be identified and marked in a constant time. Since there are $O(L)$ rows in LEFT, this work amounts to an additional $O(L)$ time.
 - (b) *Removing the redundant rows to create an upper right triangle.* Since PI is available in the beginning of the alignment stage, the $O(L)$ representative rows of LEFT can be scanned in increasing order as a first step in the alignment stage, prior to the activation of SMAWK, and the rows which become redundant identified and removed.

Altogether, this additional $O(L)$ work does not change the original $O(L)$ time complexity of the alignment stage.

5. Conclusions and open problems

The Sparse LCS Common Substring Alignment algorithm described in this paper consists of an $O(nL_y)$ time encoding stage and an $O(L)$ time alignment stage. It is intended for those applications where the

source strings contain shared and repeated substrings. Note that we just leverage on the appearance of common substrings in the source strings. It is an open problem whether a more efficient algorithm exists when the target string contains encoded repetitions as well as the source strings.

Another remaining open challenge is to try to extend the solutions presented in this paper to more general metrics, such as Edit Distance.

References

- [1] A. Apostolico, String editing and longest common subsequences, in: G. Rozenberg, A. Salomaa (Eds.), *Handbook of Formal Languages*, Vol. 2. Linear Modeling: Background and Application, Springer Verlag, Berlin, 1997, chapter 8, pp. 361–398.
- [2] A. Apostolico, C. Guerra, The longest common subsequence problem revisited, *Algorithmica* 2 (1987) 315–336.
- [3] A. Aggarwal, M. Klawe, S. Moran, P. Shor, R. Wilber, Geometric applications of a matrix searching algorithm, *Algorithmica* 2 (1987) 195–208.
- [4] G. Benson, A space efficient algorithm for finding the best nonoverlapping alignment score, *Theoret. Comput. Sci.* 145 (1–2) (1995) 357–369.
- [5] A.G. Buechner, M. Mulvenna, Discovering Internet marketing intelligence through online analytical web usage mining, *SIGMOD Record* 27 (4) (1998) 54–61.
- [6] M.S. Chen, J.S. Park, P.S. Yu, Data mining for path traversal patterns in a web environment, in: 16th Internat. Conf. on Distributed Computing Systems, 1996, pp. 385–392.
- [7] M. Crochemore, G.M. Landau, M. Ziv-Ukelson, A sub-quadratic sequence alignment algorithm for unrestricted cost matrices, in: Proc. 13th Annual ACM-SIAM Symp. on Discrete Algorithms, 2002, pp. 679–688.
- [8] D. Eppstein, Z. Galil, R. Giancarlo, G.F. Italiano, Sparse dynamic programming. I. Linear cost functions, *J. ACM* 39 (1992) 519–545.
- [9] D. Eppstein, Sequence comparison with mixed convex and concave costs, *J. Algorithms* 11 (1990) 85–101.
- [10] D. Eppstein, Z. Galil, R. Giancarlo, Speeding up dynamic programming, in: Proc. 29th IEEE Symp. on Foundations of Computer Science, 1988, pp. 296–488.
- [11] R. Farrell, P. Fairweather, E. Breimer, A task-based architecture for application-aware adjuncts, in: Proc. 2000 Internat. Conf. on Intelligent User Interfaces, ACM Press, New York, 2000, pp. 82–85.
- [12] Z. Galil, R. Giancarlo, Speeding up dynamic programming with applications to molecular biology, *Theoret. Comput. Sci.* 64 (1989) 107–118.
- [13] Z. Galil, K. Park, A linear-time algorithm for concave one-dimensional dynamic programming, *Inform. Process. Lett.* 33 (1990) 309–311.

- [14] M.S. Gelfand, A.A. Mironov, P.A. Pevzner, Gene recognition via spliced sequence alignment, *Proc. Natl. Acad. Sci. USA* 93 (1996) 9061–9066.
- [15] R. Giancarlo, Dynamic programming: special cases, in: A. Apostolico, Z. Galil (Eds.), *Pattern Matching Algorithms*, Oxford University Press, 1997, pp. 201–232.
- [16] D. Gusfield, *Algorithms on Strings, Trees, and Sequences*, Cambridge University Press, 1997.
- [17] D.S. Hirschberg, Algorithms for the longest common subsequence problem, *J. ACM* 24 (4) (1977) 664–675.
- [18] J.W. Hunt, T.G. Szymanski, A fast algorithm for computing longest common subsequences, *Comm. ACM* 20 (5) (1977) 350–353.
- [19] S.K. Kannan, E.W. Myers, An algorithm for locating non-overlapping regions of maximum alignment score, *SIAM J. Comput.* 25 (3) (1996) 648–662.
- [20] M. Klawe, D. Kleitman, An almost linear algorithm for generalized matrix searching, *SIAM J. Discrete Math.* 3 (1990) 81–97.
- [21] G.M. Landau, M. Ziv-Ukelson, On the shared substring alignment problem, in: *Proc. 11th Annual ACM-SIAM Symp. on Discrete Algorithms*, 2000, pp. 804–814.
- [22] G.M. Landau, M. Ziv-Ukelson, On the common substring alignment problem, *J. Algorithms* 41 (2) (2001) 338–359.
- [23] J. Lee, M. Podlaseck, E. Schonberg, R. Hoch, Visualization and analysis of clickstream data of online stores for understanding web merchandising, *J. Data Mining Knowledge Discovery* 5 (1/2) (2001) 59–84.
- [24] A.A. Mironov, M.A. Roytberg, P.A. Pevzner, M.S. Gelfand, Performance-guarantee gene predictions via spliced alignment, *Genomics* 51 (1998) 332–339.
- [25] A.J. Mighell, A.F. Markham, P.A. Robinson, Alu sequences, *FEBS Lett.* 417 (1997) 1–5.
- [26] G. Monge, Déblai et Remblai, *Mémoires de l'Académie des Sciences*, Paris, 1781.
- [27] E.W. Myers, Incremental alignment algorithms and their applications, Tech. Rep. 86-22, Dept. of Computer Science, Univ. of Arizona, 1986.
- [28] D. Romero, J. Martinez-Salazar, E. Ortiz, et al., Repeated sequences in bacterial chromosomes and plasmids: a glimpse from sequenced genomes, *Res. Microbiol.* 150 (1999) 735–743.
- [29] J.P. Schmidt, All highest scoring paths in weighted grid graphs and their application to finding all approximate repeats in strings, *SIAM J. Comput.* 27 (4) (1998) 972–992.
- [30] S.R. Snapp, J. Brentano, G.V. Dias, T.L. Goan, T. Grance, L.T. Heberlein, C. Ho, K.N. Levitt, B. Mukerjee, D.L. Mansur, K.L. Pon, S.E. Smaha, A system for distributed intrusion detection, in: *COMPCON Spring 91—the 36th IEEE International Computer Conference*, 1991, pp. 170–176.